Task 7

AIM:To find the length of the longest substring in a given string without repeating characters.

ALGORITHM

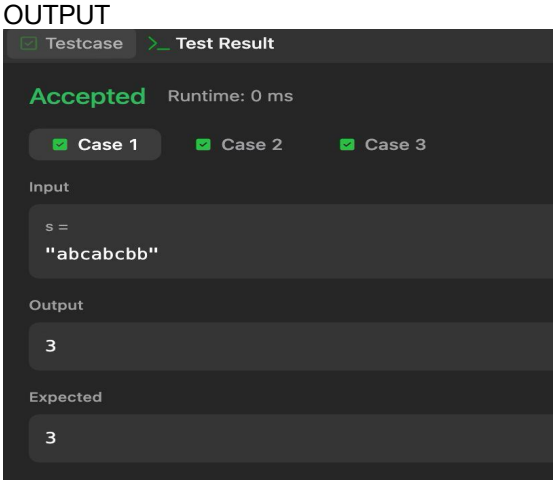
1. Initialize a HashSet to store unique characters.
2. Use two pointers: left = 0 and right = 0.
3. Traverse the string using right pointer:
4. If character is not in set → add it.
5. If duplicate found → remove characters from left until duplicate is removed.
6. Update maximum length as:
7. maxLength = max(maxLength, right - left + 1)
8. Return maxLength.

PROGRAM

```
class Solution {
   public int lengthOfLongestSubstring(String s) {
      Set<Character>set=new HashSet<>();
      int maxLength=0;
      int left=0;
      for(int right=0;right<s.length();right++){
         if(!set.contains(s.charAt(right))){
            set.add(s.charAt(right));
            maxLength=Math.max(maxLength,right-left+1);

         }else{
            while(s.charAt(left)!=s.charAt(right)){
               set.remove(s.charAt(left));
               left++;
            }
            set.remove(s.charAt(left));left++;
            set.add(s.charAt(right));
      return maxLength;}
   }
```

OUTPUT



RESULT: Thus is an program for length of the longest substring in a given string without repeating character has been executed successfully ..

Task 8

AIM:To find all words that match a given pattern based on a bijective mapping of characters.

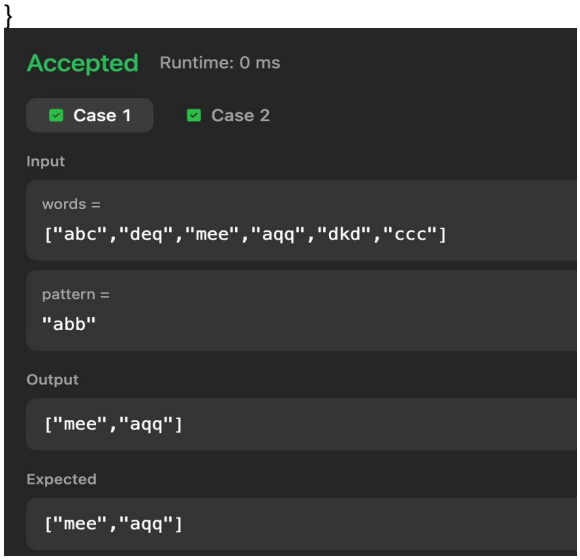
ALGORITHM

1. For each word in the list:
2. Create two maps:
3. Pattern → Word ,Word → Pattern
4. If mapping exists, check consistency.
5. If not, create mapping.
6. If mapping is valid for all characters → add word to result.
7. Return the result list.

PROGRAM

```
class Solution {
    public List<String> findAndReplacePattern(String[] words, String pattern) {
        List<String> res = new ArrayList<>();
        for (String word : words) {
            if (check(word, pattern)) res.add(word);
        }
        return res;
    }

    boolean check(String a, String b) {
        for (int i = 0; i < a.length(); i++) {
            if (a.indexOf(a.charAt(i)) != b.indexOf(b.charAt(i))) return false;
        }
        return true;
    }
}
```



RESULT: Thus is an program for all words that match a given pattern based on a bijective mapping of characters has been executed successfully

Task 9

AIM:To find all strings in an array that are substrings of another string in the same array.

ALGORITHM

1. Initialize an empty result list.
2. Compare each word with every other word:
3. If word[i] is found inside word[j] (i ≠ j):
4. Add word[i] to result.
5. Use contains() method for substring check.
6. Return the result list.

PROGRAM

```
class Solution {
    public List<String> stringMatching(String[] words) {
        List<String> result = new ArrayList<>();
        int n = words.length;
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                if(i != j && words[j].contains(words[i])) {
                    result.add(words[i]);
                    break;
                }
            }
        }
        return result;
    }
}
```

OUTPUT

RESULT: Thus an program for strings in an array that are substrings of another string in the same array has been executed successfully.